

Video 2 — Demo Agent Skill

Sinh viên 23127195 · Skill `eshop-automation` · Thời lượng ~7 phút Quay face-cam trên máy tính.

Feature demo: FR-05 — Tìm kiếm sản phẩm (*feature MỚI, chưa từng tự động hoá*)

Đề bài nói skill phải "*reusable on additional features*". Dùng skill trên một feature chưa từng đụng tới mới chứng minh được tính tái sử dụng. Nếu demo lại FR-09 thì người xem không phân biệt được skill có tác dụng gì hay chỉ đang chạy lại thứ đã có sẵn.

 **Điểm ăn tiền:** FR-05 render từ khoá tìm kiếm bằng `dangerouslySetInnerHTML` nên **có lỗ hổng XSS thật**. Em đã kiểm chứng trước: payload `<img src=x onerror="window.__xss=1">` **thực thi được**. Skill sẽ phát hiện lỗi này **ngay trên video** ở bước probe — đây không phải kịch bản dàn dựng.

PHẦN A — LỆNH COPY-PASTE

Terminal 1 — Backend (mở trước khi quay, để yên suốt buổi)

```
cd D:\Kiem_thu\HW4\HW04-TESTING
node eshop-sut/backend/database.js
cd D:\Kiem_thu\HW4\HW04-TESTING\eshop-sut\backend
node server.js
```

Terminal 2 — Frontend web (để yên suốt buổi)

```
cd D:\Kiem_thu\HW4\HW04-TESTING\eshop-sut\frontend-web
npm run dev
```

Terminal 3 — Cửa sổ sẽ xuất hiện trong video

```
cd D:\Kiem_thu\HW4\HW04-TESTING
Remove-Item -Recurse -Force test-results -ErrorAction SilentlyContinue
cls
```

Prompt gõ vào Claude Code (*Mục 3 — copy nguyên khối*)

Dùng skill eshop-automation để tự động hoá FR-05 – Product listing and search của eshop-sut.

Làm đúng quy trình trong skill:

1. Probe DOM thật trước khi viết bất kỳ selector nào
2. Dữ liệu test để ngoài file CSV/JSON trong tests/data/
3. Assertion không được race
4. Tạo tests/fr05-search.spec.js với ít nhất 12 test case
5. Chạy trên cả 3 browser

Đặc biệt kiểm tra kỹ ô tìm kiếm có lỗ hổng XSS không.

Lệnh dùng KHI ĐANG QUAY

Lệnh A — chạy 3 trình duyệt (*Mục 6*)

```
node scripts/run-multibrowser.mjs tests/fr05-search.spec.js
```

Lệnh B — mở báo cáo (*Mục 6*)

```
npx playwright show-report playwright-report/fr05-search-chromium
```

Xem xong nhấn **Ctrl + C** để tắt server report.

Checklist trước khi bấm REC

- ☐ Terminal 1 và 2 đang chạy, `http://localhost:5173` lên được
 - ☐ Chưa tồn tại file `tests/fr05-search.spec.js` — để demo từ số 0 cho thật
 - ☐ VS Code mở sẵn `.claude/skills/eshop-automation/SKILL.md`
 - ☐ Claude Code đã mở trong VS Code, sẵn sàng nhận prompt
 - ☐ Chrome mở sẵn tab `http://localhost:5173`
 - ☐ Phóng to cỡ chữ, bật webcam, tắt thông báo
-

PHẦN B — KỊCH BẢN QUAY

Mục 1 · Mở đầu — 40 giây

Hình: Face-cam toàn màn hình

Em xin chào thầy cô. Em là Trần Minh Hùng, mã số sinh viên hai ba một hai bảy một chín năm. Đây là video thứ hai của bài HW04, trình bày phần **Agent Skill**.

Ở video trước em đã demo bộ test cho feature FR-09. Video này em muốn chứng minh một điều khác: em không chỉ viết test một lần rồi thôi, mà đã **đóng gói toàn bộ quy trình làm việc đó thành một Agent Skill có thể tái sử dụng**.

Và để chứng minh nó thực sự dùng lại được, em sẽ không demo lại ba feature cũ. Em sẽ dùng skill này ngay bây giờ để tự động hoá **một feature hoàn toàn mới**, chưa từng đụng tới, đó là **FR-05 — tìm kiếm sản phẩm**.

Nếu em demo lại feature cũ thì thầy cô không thể phân biệt được skill có tác dụng thật hay em chỉ đang chạy lại thứ đã viết sẵn.

Mục 2 · Giới thiệu skill — 1 phút 40

Hình: VS Code → `.claude/skills/eshop-automation/SKILL.md`

Đây là file skill. Về bản chất, một Agent Skill là một file hướng dẫn mà AI sẽ tự động nạp vào khi gặp đúng loại công việc.

Điểm em muốn nhấn mạnh nhất: file này **không phải lý thuyết chép từ tài liệu Playwright**. Nó là kết tinh từ những lỗi mà **chính em đã thực sự mắc phải** khi làm ba feature trước. Mỗi quy tắc trong đây tương ứng với một lần em phải ngồi debug mất công.

(Cuộn tới bước 1 và bước 2)

Bước 1: đọc đặc tả trước khi đọc mã nguồn. Lý do là nếu ta đưa mã nguồn cho AI trước, nó sẽ viết test khớp với hành vi hiện tại của chương trình. Loại test đó luôn luôn xanh và **không bao giờ tìm ra lỗi** — nó biến bug thành đặc tả. Đây là cái bẫy lớn nhất khi dùng AI để viết test.

Bước 2: phải dò DOM thật, tuyệt đối không được đoán. Trong hệ thống EShop này, hàm `getByLabel` — là cách chuẩn nhất để tìm ô nhập liệu — **không bao giờ hoạt động**, vì các thẻ label không gắn thuộc tính `htmlFor`. Riêng lỗi này em đã mắc **hai lần**: một lần ở trang đăng ký, và sau khi đã sửa rồi vẫn mắc lại lần nữa ở trang quản trị. Nên em ghi hẳn vào skill để không bao giờ mắc lần thứ ba.

(Cuộn tới bước 4)

Bước 4 ghi quy tắc quan trọng nhất về assertion: **không bao giờ chứng minh "việc gì đó không xảy ra" bằng cách kiểm tra URL chưa thay đổi**. Vì ngay sau khi bấm nút, ứng dụng chưa xử lý xong nên URL đương nhiên chưa đổi, và câu lệnh kiểm tra sẽ khớp ngay lập tức rồi chuyển xanh. Đây chính là lỗi đã khiến bốn test của em báo xanh giả, mà em đã kể ở video trước.

(Cuộn tới bước 7)

Bước 7: con số không phải là bằng chứng. Có một lần bộ test của em cho ra đúng con số kỳ vọng, nhưng khi em đọc kỹ thông báo lỗi thì phát hiện năm test đang đỏ vì một lý do **hoàn toàn khác** với lỗi mà chúng nhắm tới — do một hàm phụ trợ bị hỏng. Nếu chỉ nhìn tổng số pass và fail thì em đã kết luận sai hoàn toàn.

(Cuộn xuống bảng "SUT facts")

Và cuối file là bảng những sự thật về hệ thống mà em đã tốn công phát hiện: mật khẩu admin thật là `Admin123!` chứ không phải như tài liệu hướng dẫn ghi; cơ chế khoá tài khoản kích hoạt chỉ sau **hai** lần nhập sai chứ không phải ba; Vite chỉ lắng nghe trên IPv6 nên phải dùng `localhost` chứ không dùng được `127.0.0.1`; giỏ hàng không lưu trữ nên tải lại trang là mất.

Lần sau bất kỳ ai dùng skill này sẽ không phải mò lại từ đầu.

Mục 3 · Gọi skill — 40 giây

Hình: Claude Code trong VS Code — dán prompt ở Phần A

Bây giờ em gọi skill. Thầy cô để ý: em **không mô tả lại quy trình**. Em chỉ nêu tên skill và tên feature cần làm, cộng thêm vài gạch đầu dòng nhắc lại yêu cầu của đề. Toàn bộ hướng dẫn chi tiết đã nằm sẵn trong file skill rồi.

(Đợi Claude Code nạp skill, chỉ vào dòng báo skill được nạp)

Đây, skill `eshop-automation` đã được nạp vào. Từ giờ nó sẽ làm theo đúng tám bước trong đó.

Mục 4 · ★ Bước probe phát hiện XSS — 1 phút 20

Hình: Claude Code đang viết và chạy script probe

Đúng như skill quy định ở bước 2, việc đầu tiên nó làm **không phải là viết test case**, mà là viết một đoạn script tạm để dò xem trang tìm kiếm thực sự có những gì: ô nhập liệu tên là gì, nút bấm nhấn ra sao, và quan trọng nhất là hệ thống phản ứng thế nào với các đầu vào đặc biệt.

Nhiều người khi dùng AI sẽ bỏ qua bước này cho nhanh. Nhưng đây chính là bước em cho là đáng giá nhất.

(Khi kết quả probe hiện ra, trở vào dòng có `__xss`)

Và đây là kết quả. Thầy cô nhìn dòng này: từ khoá tìm kiếm được trang web render bằng `dangerouslySetInnerHTML`. Đây là một hàm của React cho phép chèn HTML thô vào trang mà **không lọc gì cả**.

Cho nên khi script probe nhập vào một payload tấn công XSS, cụ thể là một thẻ `img` với thuộc tính `onerror`, thì **đoạn mã đó đã thực thi thật** — biến `window.__xss` được gán giá trị bằng một.

Đây là một lỗ hổng **Cross-Site Scripting** thật sự. Kẻ tấn công có thể gửi cho nạn nhân một đường link chứa payload, và khi nạn nhân mở link thì mã độc chạy trong phiên đăng nhập của họ, đánh cắp được token hoặc cookie.

Điều em muốn nhấn mạnh: nếu làm theo cách thông thường là viết test ngay từ mô tả feature, em sẽ chỉ kiểm tra "gõ từ khoá vào có ra đúng sản phẩm không" và **bỏ sót hoàn toàn** lỗi bảo mật này. Chính vì skill bắt buộc phải probe trước nên nó mới lộ ra.

(Chuyển sang Chrome, tự nhập payload vào ô tìm kiếm để chứng minh)

Em kiểm chứng lại bằng tay cho chắc chắn.

Mục 5 · Skill sinh data file và spec — 1 phút

Hình: Claude Code tạo `tests/data/fr05-*` rồi `tests/fr05-search.spec.js`

Sau khi đã biết chắc hệ thống hoạt động ra sao, lúc này nó mới bắt đầu sinh test.

Theo bước 3 của skill, toàn bộ dữ liệu kiểm thử được đưa ra file CSV và JSON riêng trong thư mục `tests/data`. Đề bài cấm viết cứng dữ liệu trong file spec, vi phạm là bị loại.

(Mở file spec vừa được sinh ra, cuộn qua vài test case)

Trong file spec, các assertion đều được gắn nhãn từ P1 đến P6 ngay tại chỗ dùng, để người chấm dễ đối chiếu với yêu cầu tối thiểu ba kiểu assertion của đề.

Thầy cô chú ý test kiểm tra XSS: nó **không** kiểm tra gián tiếp qua URL hay qua chữ hiển thị, mà kiểm tra **trực tiếp** xem đoạn mã độc có được trình duyệt thực thi hay không. Đây đúng theo nguyên tắc "assertion không được race" ở bước 4 của skill.

Mục 6 · Chạy 3 browser và xem báo cáo — 1 phút 30

Hình: Terminal 3 — dán **Lệnh A**

```
node scripts/run-multibrowser.mjs tests/fr05-search.spec.js
```

Nói trong lúc chờ:

Bước 6 của skill: chạy trên cả ba engine, mỗi engine sinh một báo cáo HTML riêng.

Thầy cô có thể thấy: toàn bộ quá trình từ lúc em gõ **một câu prompt** cho tới khi có một bộ test hoàn chỉnh, chạy được đa trình duyệt, chỉ mất khoảng vài phút.

Nếu không có skill, em sẽ phải mô tả lại từ đầu tất cả những quy tắc kia trong mỗi lần làm việc. Mà như em đã kể, **chính em đã mắc lỗi selector đúng hai lần** trên hai màn hình khác nhau — nghĩa là chỉ nhớ trong đầu thì không đủ, phải ghi thành văn bản để công cụ tự động áp dụng.

*(Khi chạy xong, dán **Lệnh B**)*

```
npx playwright show-report playwright-report/fr05-search-chromium
```

(Trò vào tiêu đề báo cáo)

Báo cáo có đầy đủ dòng **Run by: 23127195** kèm timestamp ISO, y hệt ba feature trước. Lý do là cấu hình đó đã nằm sẵn trong file `playwright.config.js` của dự án, và skill biết cách tận dụng lại thay vì tạo mới.

Mục 7 · Kết — 40 giây

Hình: Face-cam

Em xin tổng kết phần Agent Skill.

Skill này biến quy trình mà em rút ra được sau ba feature thành một thứ **dùng lại được ngay cho feature thứ tư**, mà em không phải nhớ lại hay tra cứu lại gì cả.

Điểm em tâm đắc nhất là nó không chỉ ghi *phải làm gì*, mà quan trọng hơn là ghi **vì sao phải làm như vậy**. Mỗi quy tắc đều gắn với một lỗi thật mà em đã mắc, nên người đọc hiểu được lý do chứ không phải học thuộc máy móc.

Và như thầy cô vừa chứng kiến, ngay trong lần sử dụng đầu tiên trên một feature hoàn toàn mới, nó đã giúp em phát hiện thêm **một lỗ hổng XSS** mà nếu làm theo cách thông thường thì chắc chắn em đã bỏ sót.

Em xin chân thành cảm ơn thầy cô đã theo dõi cả hai video.

PHẦN C — KIỂM TRA SAU KHI QUAY

- ☐ Thấy rõ skill được **nạp và sử dụng thật** (không phải chỉ mở file ra đọc)
- ☐ Thấy đủ chu trình: **probe** → **data file** → **spec** → **chạy 3 browser** → **report**
- ☐ Có cảnh phát hiện **lỗ hổng XSS** và kiểm chứng lại bằng tay
- ☐ Báo cáo hiện **"Run by: 23127195"** + ISO timestamp
- ☐ Upload YouTube → chọn **Unlisted**
- ☐ Dán link vào `23127195/README.md`
- ☐ **Nếu FR-05 sinh ra bug mới** → **tạo GitHub Issue #16 cho lỗi XSS**

PHƯƠNG ÁN DỰ PHÒNG

Nếu lúc quay Claude Code gặp trục trặc hoặc mạng chậm, chuyển sang phương án an toàn: quay lại quá trình dùng skill trên **FR-14** (feature đã làm xong), vừa chạy vừa đối chiếu từng bước với file `SKILL.md`.

Video vẫn hợp lệ vì đề chỉ yêu cầu *"shows how you used the skill on a complete feature"* — nhưng sức thuyết phục kém hơn hẳn so với làm trên feature mới.